BSc.(Hons) in Software Engineering

Faculty of Computing

Reducing vehicle waiting times at traffic lights using camera-based traffic monitoring

Name : **W.G.B.M.Priyankara**

Student Registration Number : **IT_IFLS_001/B003/0037**

Student Number : **M20001004005**

Module : **IT 4116 - Research Methods**

Submission Date : **13/11/2025**

Table of Contents

Table of Figure

Chapter 1: Introduction

In the current state of rising urbanization and urban growth, road transport infrastructure in major urban centers is struggling to meet the rising number of owners of vehicles. Road transport infrastructure in urban centers is becoming more congested, especially at major junctions such as four-way intersections. In the past, these junctions were managed by traffic light control systems that rely on fixed timing. While these systems worked properly, they were "blind" to what was going on the road.

In most instances, road users find themselves waiting at a red traffic light with no indication of approaching vehicles on the road intersection. The ineffective system is responsible for various negative effects, such as unnecessary time delays, high fuel consumption, high air pollution resulting from idling, and high driver dissatisfaction. However, with the use of low-cost cameras, as well as sophisticated computer algorithms, this is now a revolutionary method. From fixed timing to flexible activation, there is now the ability to apply the use of visual information in current road intersections. The objective of this study is the development of a prototype system capable of utilizing visual recognition of vehicles with sophisticated control algorithms to mitigate idling delays.

1.1 Background and Motivation

The basic flexibility of current traffic control systems is used as a premise in this study. Current systems operate on predetermined cycle times, often based on actual counts or traditional notions of peak hour volume. Such "open-loop" systems are not capable of accounting for the randomness of actual traffic patterns, influenced by uncertainties of actual traffic unpredictable events such as weather conditions, road incidents, or spontaneous changes in commuter behavior.

If these fixed-time signals do not correspond to real-time demands, then the “dead time” during the green light for the empty lane becomes more than just a nuisance; it becomes a measurable delay. This disparity between signal time and actual demands continues to deteriorate:

- Macro-economic Costs: There would be a loss of productive work-hours due to longer average waiting times.
- Environmental Degradation: Idling contributes to higher emissions of CO2 and particulate materials in the environment, resulting in an urban heat island effect and adverse health conditions.
- Resource Inefficiency: Resource inefficiency results in fuel consumption and cost strain on vehicle owners or drivers.
- Safety Risks: Waiting times may induce tension in drivers, resulting in aggressive driving practices, such as exceeding red lights.

Because cameras are major sensors, they help us build an affordable and flexible solution. Also, they differ from inductive solutions installed beneath asphalt layers since they are easy to install and maintain. The cameras offer an overarching "top-down" view of the intersection, enabling sophisticated data retrieval, surpassing mere presence detection.

1.2 Problem in Brief – Scope

The main problem solved in this particular project is the lack of responsiveness of the existing traffic infrastructure, which causes a waste of available road capacity. The proposed study considers only a four-way junction model with four different camera modules that observe each incoming approach individually.

The extent of implementing it comprises:

- Computer Vision Integration: Utilizing digital images for detecting cars in high-definition videos.

- Temporal Validation: Employ "demand validation" logic to detect a vehicle within a 5-second time frame, rejecting any noise or non-vehicle movement.
- A priority algorithm was designed for an equal distribution of traffic by giving a green signal to the first route that records a corresponding need. (First-Come, First-Served Logic).
- Hardware Actuation: Automated transformation of the digital signals for detection into light positions (Red, Yellow, Green) using a micro-controller interface.

1.3 Aim and Objectives

➢ Aim

Indeed, the aim and object of this research is to design and implement an operating hardware-software prototype for an adaptive camera-based real-time traffic signal control system. This is with the view to reduce vehicle wait times at intersections from dependence on static timing signals to demand-related signals.

➢ Objectives

From the preceding overview, specific objectives stated to realize the above-mentioned aim include:

- Literature Synthesis: Review the state of the art in traffic control and computer vision-based techniques for vehicle recognition.
- Analysis of constraints: Numerically and qualitatively analyze the constraints of fixed-time systems operating in an urban setting.
- System Architecture Design: Provide a robust framework of a real-time monitoring system for a four-way intersection.

- The algorithm for vehicle detection should be developed, having a validation window of 5 seconds to ensure system dependability.
- Decision Logic Design: Implement a priority-based arbiter that grants right-of-way depending on order of arrival.
- Prototype implementation: Implementation of physical system with Arduino microcontroller, LED indicators, and camera modules.
- Validation and Testing: Test the efficacy of the prototype's reduction of idle time using simulated traffic scenarios against standard methods.

1.4 Proposed Solution Overview

The proposed system is an intelligent gateway between digital perception and physical control. The system architecture is divided into three functional layers:

- The Input Layer relies on four camera modules (ESP32-CAM and USB webcams) that capture video feeds of the road intersections. The camera modules act as the system's eyes that supply the raw visual data to the processing unit.
- The Processing Layer employs Python and OpenCV for video frame processing at a rate of 10-30 FPS. The system identifies vehicles and their contours as they appear and disappear within the detection zone. When a vehicle stays inside the detection zone for 5 seconds, a “Green Light Request” message is created and sent to the hardware controller through a serial interface.
- The Output Layer: Arduino Uno/Mega will receive the command and will transfer the signal. It will control the safety intervals (the yellow light transition) and will update the LED light signals corresponding to traffic lights.

- **Important Features:**

- Demand-Based Validation: A 5-second timer eliminates "false positives" due to shadows or pedestrians.
- Hardware-Software Synergy: It integrates the high-level programming of Python with the reliable low latency of Arduino.
- The "Smart City" is cost-effective and uses off-the-shelf components, thereby making it a practical choice for urban development.

1.5 Layout of the Final Report

The report is systematically organized into six chapters to give a comprehensive history of the entire research process:

- Chapter 1 (Introduction) provides the foundation of the study background and major objectives.
- Chapter 2 (Literature Review) - Analyzes current literature on traffic management, image processing, or computer vision methods.
- Chapter 3 (Technology Description) provides information about the hardware components (Arduino, Cameras) and software (Python, OpenCV) used for the project.
- Chapter 4: Methodology Detailed explanation of the implementation phase, including the detection logic and the priority switching technique.
- Chapter 5 (Analysis and Design) contains the experimental design, testing results, as well as analysis of results.
- Conclusion (Chapter 6) summarizes the accomplishments of the project, points out limitations, and suggests ways for scaling the system in the future.

1.6 Summary

This chapter deals with solving the problem associated with urban traffic congestion, which results from traditional traffic signals that use fixed times despite being "blind to real-time road conditions". This results in additional delays, fuel consumption, and environmental damage. This research work proposes a cost-effective camera design to change from fixed to demand-driven traffic signals using computer vision and Arduino controllers. The research work deals with a four-way intersection requiring a 5-second "demand validation" logic to eliminate noise and a "First-Come, First-Served" algorithm to ensure equal traffic share.

Chapter 2: Review Of Others' Work

2.1 Introduction

This has led to a corresponding increase in the growth of vehicle density, which has placed tremendous pressure on the now largely inadequate existing road systems. Traditional fixed-cycle timer-based traffic control systems are now increasingly viewed as a major hindrance to urban movement. In other words, these "static" systems, devoid of real-time sensory capability, come out with systemic inefficiencies such as increased idle time, fuel wasted, and higher greenhouse gas emissions.

The academic and engineering community started focusing on ITMS to find solutions for these issues. This chapter tries to give a comprehensive overview of technological advances in this subject, mapping the route from early algorithmic approaches to the current integrations of IoT, CV, and Machine Learning in state-of-the-art solutions. Also, by making a review of these developments, we could have a better position about the current situation and indicate precisely which points adaptive signal control can make more of an impact.

2.2 Early Systems Based on Image Processing

Before the acceptance of modern artificial intelligence in general acceptance, investigations were upon the fundamentals of image processing methods to translate visual information directly into usable traffic information. The "classical methods" were mainly engaged in identifying movements and presence at the pixel level.

- The earlier systems employed methods such as Background Subtraction, involving comparison of a static image of an empty road with live images to detect "foreground" objects such as cars, while other methods for approximating traffic speed and direction were Frame Differencing and Optical Flow.

- To address this issue, Almawgani [2] proposed a MATLAB-based prototype that included an Arduino controller to analyze pixel intensity. Based on the analysis of pixel intensity variance in Regions of Interest, this approach classified traffic density into "Low" and "High" categories. The weakness of this strategy was that it was not very robust, meaning it was very sensitive to environmental conditions such as camera shake, shadow movement, or illumination. Despite this, the approach had an accuracy of 86%.
- Sonnleitner observed similar phenomena using CCTV-based surveillance to estimate congestion levels [7]. The result showed a basic weakness of motion-based detection: not being able to detect stationary cars (e.g., cars stopped at red lights), which eventually integrate into the background model.
- In South America, a study by Ximenes et al. [10] explored the concept of picture segmentation at a low cost. The research achieved a level of 83% accuracy with a focus on region-based pixel change, emphasizing that although traditional image processing is possible, it is often imprecise for critical urban traffic management systems.

2.3 Adaptable Frameworks with IoT Integration

The development of Internet of Things (IoT) has taken the concept of traffic control from isolated crossroads to a network and data-driven environment. The integration of IoT made it possible to adopt a multi-sensor approach that uses visual and telemetric information.

- e-Prime publication showed that sensor fusion works well in multi-modal sensing environments. The number of vehicles and speed could be measured with high fidelity using a combination of camera views and ultrasonic sensors installed along the edge of the road, a two-layered verification approach which achieved a 25% reduction in traffic congestion.
- Khedkar et al. [3] were the pioneers in utilizing Raspberry Pi as a processing node at the edge level. The team created a communication architecture that

utilized the MQTT (Message Queuing Telemetry Transport) protocol to quickly transmit traffic information to a message center. The modification to the architecture saw a 30% boost in traffic throughput, thereby proving that processing at the edges reduces latency charges.

- A distributed AI network was proposed by Zrigui et al. [11]. Here, instead of regulating intersections individually, this system enables neighboring crossings to "speak" to each other, exchanging density information so as to coordinate green waves on an entire region.

2.4 Tracking and Detection Based on Deep Learning

A major technological breakthrough in traffic came with the introduction of Convolutional Neural Networks. Unlike other traditional methods of image processing, Deep Learning lets the system "learn" the properties of a car (wheels, windshields, and lights), making its detection much more accurate.

- Abbas et al. [1] employed an improved version of the R-CNN, with the ability for high-precision target detection. This very model was able to achieve a remarkable 95% detection accuracy with the ability to retain performance under the worst weather conditions, usually fatal to classical systems, such as heavy rain and fog.
- Wieczorek et al. [9] proposed multi-scale CNN architectures to deal with the problem that vehicles appear differently because of distance from the camera; that is, this would ensure small background objects were detected as accurately as larger foreground objects.
- The industry is moving to YOLO models for contemporary real-time architectures. In this work, Liu et al. [5] combined YOLOv8n with ByteTrack, which is a tracking method that provides a unique ID to each vehicle detected. This system removes "double counting" and led to improving the overall accuracy by 4–7%.

- In Teslyuk et al., [8] YOLO-based frameworks were utilized to compute precise lane density. Feed such density measures as an input to their control logics; authors managed to reduce the average wait times by 25%, which is an example of the power of merging high-accuracy detection with adaptive algorithms.

2.5 Real-Time Control and Optimization Strategies

Detection of a vehicle is just half the problem; then comes the need to compute the optimal allocation of “Green Time”. It involves intricate logic in decision-making processes.

- Abbas et al. [1] and Sahu et al. utilized proportional control systems. In these models, the time allocated for a green signal is proportional to the number of vehicles detected in one lane compared to another.
- Kutlimuratov et al. [4] analyzed Fuzzy Logic controllers in the context of heuristic and intelligence logic. Traffic is rarely "black and white"; Fuzzy Logic allows the system to process unclear expressions like "moderately congested" or "slightly empties," resulting in smoother transitions between signals.
- Teslyuk et al. [8] analyzed models of Reinforcement Learning (RL), where traffic managers learn based on trials and error. The system identifies optimal timing cycles for each time of day through feedback loops.

2.6 Research Gaps and Problem Highlight

While interest in this field of research is enormous, there are some critical gaps that prevent these technologies from being taken up completely by emerging urban environments.

- ❖ Simulation vs Reality: Most research is performed within simulated environments, such as SUMO or MatLab. What is lacking are physical hardware prototypes that can actually put these notions to test in the real world.

- ❖ Expensive GPUs are usually required for high-end Deep Learning models, which is a hardware constraint. Therefore, from an economic viewpoint, these models are not viable for mass usage in developing countries.
- ❖ Latency in Live feeds: Low-power edge devices with HD feeds usually observe latency in detection and actuation.
- ❖ Local vs Global Optimization: Most research focuses on either a single junction or a city-wide network; few mid-tier solutions exist that integrate local adaptive logic with cost-effectiveness.

- ➢ **Highlighted Problem:**

The current scenario faces a paradox; the solutions are either very rudimentary in their approach to be termed right or computationally complex to be of any use. This research work intends to bridge this divide. The paper proposes a real-time solution that can utilize a light YOLOv8 algorithm for detection and a rule-based adaptive control algorithm designed in an Arduino that can offer an efficient, highly precise, yet affordable solution in keeping with the burgeoning requirements of the traffic in cities like Sri Lanka.

2.7 Summary

This chapter reviews the history of traffic management from pixel-level image processing in the early days to AI and IoT integration in the current period. These early days were based on background subtraction but had problems related to stationary vehicle recognition and weather sensitivity. These issues were overcome by better models based on the IoT framework for network coordination and deep learning techniques (Faster R-CNN and YOLOv8) that provided highly efficient results in various weather conditions. The author identifies the research gaps by explaining that current methods are either too simple or require costly hardware – in this case, providing an answer to the apparent "paradox."

Chapter 3: Technology Adapted

3.1 Introduction

The successful implementation of a camera-based adaptive traffic management system requires a seamless integration of high-performance computational vision with hardware execution that has low latency. The need for a system that can withstand complex visual data in a real-time processing mode yet remain affordable enough in terms of being a viable alternative for towns in developing countries defined the technology choices in building this prototype.

This chapter provides a complete overview of the hardware components, software environments, and data handling techniques involved in the study. The technical characteristics of the tools considered are recorded, providing a platform for the next approach and experimental design to be encountered.

3.2 Hardware Specifications

The physical prototype is made using industrial-standard parts that have been chosen on the grounds of reliability, easy integration, and fast response time. Its hardware design is flexible and allows the monitoring of each of the four paths at the intersection independently.

- **Arduino Microcontrollers (Uno or Mega)**

Arduino is basically the "control center" of the system. Even as the intensive computations are processed by the computer, the Arduino is the one entirely controlling the final physical settings of the traffic lights.

- Roles: It receives commands based on logic through a serial cable and responds to the time sequences corresponding to Red, Yellow, and Green light situations.

- Reasoning: The Arduino Uno/Mega was selected because of its deterministic model that is capable of operating timing-critical code in the background as opposed to older operating systems.

➢ Camera Modules (USB Camera / ESP32-CAM)

There are four independent camera modules, and these are responsible for the acquisition of visual data, with each of them located at strategic positions to monitor an approach lane.

- Role: These modules function as a means of capturing active video streams with a resolution adequate enough for car detection (normally 640×480 or better) and transmitting frames to a processing unit at a speed of 10-30 frames per second.
- Justification: USB-based or ESP32-CAM modules provide an effective scaling alternative compared to high-end traffic cameras available on the commercial market, and thus this system can be widely tested in an urban environment.

➢ Traffic Light Indicators and Circuitry

Visual signaling in the prototype is depicted by a set of Light Emitting Diodes (LEDs).

- Components: High-brightness Each of the four paths employs LEDs of colors red, yellow, and green.
- Support Hardware: Breadboards are employed in prototyping, incorporating resistors that protect LEDs from current overload. The system requires a good power source of 5V or 12V to power the sensors and control boards in a manner that avoids fluctuations in voltages during signal switching.

3.3 Software Environment

This software component is referred to as the “brain” and will be used to recognize complicated patterns and translate those visual patterns into hardware-executable commands.

- **Python Programming and OpenCV**

The programming language used will primarily be Python, which has been chosen for its vast libraries for data science and automation.

- OpenCV (Open Source Computer Vision Library): It is the core engine for the real-time processing of the picture. OpenCV is applied in the project for the manipulation of the frames of the picture. It is applied in the conversion of the color space of the picture as well as the cropping of the Region of Interest (ROI).

- **YOLOv8 Algorithm (You Only Look Once)**

The YOLOv8 model is employed to enable detection capabilities for the system above. The model works differently compared to conventional sliding window approaches because it scans an entire image only once to achieve phenomenal speed with superior accuracy.

- Implementation: In this particular project, a light version (YOLOv8n) is designed and altered to make sure that this process can function on a normal consumer computer or even on an edge computer. Based on that, it is capable of recognizing and detecting vehicles (car, lorry, and motorbike) and providing the control logic with the data required for occupancy.

➢ Arduino IDE and C++ Integration

The Integrated Development Environment (IDE) used for creating firmware for microcontrollers is Arduino IDE. This code is written in Embedded C++. One area of emphasis is given to the implementation of state machines that will determine safe transitions for traffic lights, including delays like yellow.

➢ Serial Communication (pyserial)

The pySerial library is an essential part of this system as it acts as a communication bridge between the Python script and the Arduino hardware.

- Process: Once the Python software recognizes the car within the required 5-second time frame, it transmits a predetermined byte code to the USB serial port. The Arduino receives these bytes, with rapid execution of the necessary signal alteration.

3.4 Dataset and Data Handling

In contrast to most research work, which uses static and pre-labeled datasets (like COCO or ImageNet) for model training, our research is centered around the processing of dynamic real-time data.

➢ Real-Time Video Acquisition

The primary functionality of the system does not depend on the video file storage. Rather, it processes real-time video streams from four different camera angles concurrently. This verifies that the decisions made by the system will be based on the current state of the intersection and the actual traffic demand.

➢ Temporal Validation (5-Second Rule)

To provide integrity for the data, the system involves a logic of temporal validation. The “vehicle detection” event is taken to be valid only when there is a constant observation of the vehicle within a specific detection zone for 5 seconds. This ensures

that the system does not act upon “noise” events, such as a car passing through an intersection or a camera malfunction.

- **Performance Metrics and Analytics**

- The system will be set up in such a way as to log certain performance metrics during its testing and assessment phases for further analyses.
- Detection Latency: Time difference between the arrival of a vehicle and its successful identification.
- Signal Transition Delay: Time difference between a Python instruction and the actual changing of state in the LEDs.
- Throughput Statistics: Compared the number of cars removed in every cycle between adaptive and fixed-time simulations.

The data from this will help the team to measure the efficiency gains of the camera-based technique over previous methods.

3.5 Summary

This chapter describes the hardware and software stacks required to achieve high-performance vision with low latency actuation. On the hardware side, this includes Arduino microcontrollers for final signal settings, USB or ESP32-CAM for real-time video collection, and high-brightness LEDs for signaling. Software is developed in a Python environment, using the OpenCV library for image manipulation. The lightweight YOLOv8n model used herein allows for fast vehicle recognition and lane occupation analysis. The critical connection is provided by the pySerial package, which ties Python's detection logic to the Arduino's deterministic execution of light sequences.

Chapter 4: Approach

4.1 Introduction

Implementation aspect of this research project involves computer theory as well as engineering implementation. The prime challenge being tackled in this chapter includes converting raw visual information into a structured predictive control system. This chapter also includes the procedure followed for developing the detection system, refining logic for decisions based on priority, and integrating the hardware module developed on Arduino. The aim is to develop an adaptive system for reacting to real-world changes in traffic conditions to ensure that green light allocation is done according to actual need rather than predicted values.

4.2 System Architecture and Workflow

The proposed system incorporates a multilevel architecture that differentiates between high-level perception and low-level execution tasks. This allows for ease of debugging within the system since it is scalable.

- **Data Acquisition and Pre-processing**

All these process steps initiate from the four intersection approaches where USB or ESP32 CAM modules act as main sensors. The cameras are strategically placed to offer a good view of the "Stop Line" as well as the approach lane nearest to it. The video is captured and fed into the Python processing environment at a constant frame rate of 10-30 frames/second, which is high enough to analyze the movement of vehicles.

- **The Processing and Logic Tier**

Once the frames are received, control passes to the Python processing algorithm. Environmental noise reduction work, such as converting the picture to grayscale and applying a Gaussian filter, is done with the help of OpenCV. The final decision for the whole intersection is made by calculating the lane occupancy based upon a priority algorithm.

- **Hardware Actuation**

The last step of the process is the realization of logic through a physical expression. Control commands are serialized for transmission to the Arduino microcontroller. The Arduino is basically a dedicated state machine that processes electrical signals to light up LED indicators for traffic lights.

4.3 Implementing the Detection Mechanism

The accuracy of the adaptive system is completely reliant on its ability to distinguish a genuine car from ambient noise.

- **Region of Interest (ROI) Mapping**

Instead of analyzing the whole frame of video, which is highly computationally intensive, this system picks out distinct Regions of Interest. These "digital zones" are then laid on top of the entrances to the four lanes. It only has to search these ROIs look for changes in pixel density or shape motion, making it much faster.

- **Real-Time Detection and Background Subtraction**

The detection phase involves background elimination and shape recognition. The system is able to distinguish the shapes of the approaching automobile by comparing the actual image with the model of the empty road. If the shape corresponding to the predefined size limit is detected in the ROI region, the detection flag is set.

- **The 5-Second Validation Window**

Reliability can be achieved at a high level by the use of a time-validation filter. A car needs to be recognized correctly within the ROI for at least 5 seconds.

- The technology does not take into account any cars passing through the area in a second and therefore prevents triggers caused by people or cars coming from the other side.

- Only stopped or sufficiently slowed-down vehicles are considered "in demand" similar to inductive loops but with visual verification.

4.4 Priority-Based Traffic Light Control Algorithm

The software realization is based on the Priority-Based Control Algorithm that defines transitions at right-of-way points.

- **Demand-Driven Switching**

In contrast to fixed-cycle timers, which continue rotating traffic signals independent of traffic conditions, the design of this project is demand-driven. If none of the four roads' traffic is detected, then the status would be "Red-All" or Default State (for example, Road 1 would be Green). The system begins a traffic change cycle when it exceeds the corresponding 5-second timer for a particular lane's verification timer.

- **First-Come, First-Served (FCFS) Priority**

For the purpose of fairness, a sequential priority system is adopted in this approach. The first road on which a verified car will pass will receive the green signal. If several cars arrive simultaneously on different roads, the system will queue these cars in the order of their verification.

- **Interlock Safety Protocol**

It is essential for safety at junctions while organizing traffic flow. Once the computer identifies that Road A should get a green light, it automatically starts an "Interlock" process. This ensures that no other roads (B, C, and D) are left in green when Road A gets its green light. This eliminates problems associated with multiple green lights due to improper logic.

4.5 Hardware Integration and Serial Communication

The final stage in the process of implementation is dubbed 'Handshake' and is where Python programming meets Arduino hardware

➢ The Arduino State Machine

Arduino is integrated with a custom script that decodes single-byte commands. It includes specific safety time intervals that would be hard to handle on its own using the Python script, like the 3-second yellow light interval. As the command is sent from the Python script to change the signal light state, the Arduino will handle the sequencing of the time as follows:

- Current Green Lane → Yellow (3 Seconds)
- Current Green Lane → Red
- Target Lane → Green

➢ Serial Link and Command Mapping

Using the pyserial library, the Python script communicates at a baud rate of 9600. The command dictionary is used to ensure accuracy:

- Command 'A': Logic for Road 1 Priority.
- Command 'B': Logic for Road 2 Priority.
- Command 'C': Logic for Road 3 Priority.
- Command 'D': Logic for Road 4 Priority.

Such an implementation technique allows for smooth, low latency, and secure transition from digital traffic detection to physical traffic control.

4.6 Summary

This chapter goes over the procedural implementation of the system, dividing it into high-level perception in Python/OpenCV and low-level execution in Arduino. For efficiency in processing, the system utilizes ROI mapping instead of analyzing complete video frames. A validation window of 5 seconds is utilized to validate actual vehicle demand but prevents spurious triggers from pedestrians or passing oncoming/offcoming traffic. Accompanying the logical processing is a priority scheme

of "First-Come, First-Served" and an "Interlock" for safety, ensuring that only one lane gets a green light at any one time to avoid signal conflicts.

Chapter 5: Analysis And Design

5.1 Introduction

The transition from a workable prototype to a validated solution requires exhaustive performance testing under a range of scenarios. The objective of this chapter is to discuss an experimental framework for evaluating the effectiveness, reliability, and efficiency of the camera-based adaptive traffic signal system. The analysis is meant to make available a comparison between modern demand-driven logics against traditional fixed-time control approaches. The present chapter attempts to quantify gains in urban mobility and responsiveness of traffic signals by the proposed system through the development of a controlled test environment using selected standards for performance evaluation.

5.2 Study Design and Hypothesis

The research uses an empirical experimental design that relies on a prototype of a four-way intersection in both real form and digital format. The experimental design aims to evaluate the system's ability to minimize "dead time" when a road has a lone green light with no traffic, yet the other lanes are packed.

- **The research is guided by three primary hypotheses:**

  - Efficiency Hypothesis: A camera-based detection system can substantially decrease the mean time that vehicles have waited compared with traditional fixed-time signals.
  - Accuracy Hypothesis: OpenCV and YOLOv8 integration will result in a detection accuracy of over 90% in a laboratory
  - Hardware Reliability Hypothesis: It is hypothesized that after receiving an instruction from the Python processing layer, the Arduino-based hardware interface is capable of changing signals with less than a second’s delay.

5.3 Setting of the Study

For validation of data authenticity, this research work has been performed in two differing settings with a particular analysis goal in each scenario.

- **Laboratory / Indoor Testing Environment**

The physical prototype was built under a controlled lab setting. The lab setting played a significant role during the integration of hardware and software components. By utilizing a physical prototype, the research team could make alterations related to lighting effects and the focal length of the camera, as well as the location of the “Stop Line” region of interest. This option offered a baseline for evaluating the physical response time of LED indicators as well as the reliability of the serial data channel.

- **Computer Simulation and Visualization Environment**

Additionally, in order to complement the physical simulation environment, a computer simulation environment has been developed in Python graphics. Even though the physical simulation environment is the best for illustrating the concept or idea of the system, simulation on the computer allows for the simulation of patterns involving large volumes of traffic as well as the concept of "worst-case scenarios," things that would not easily be replicated in the mini models.

System Phases and Experimental Setup

The project broke down the development life cycle into a modular form consisting of four phases, which ensured a systematic enhancement of the system's functionality.

- **Phase 1: System Development and Calibration**

The first stage focused on the interface of software and hardware components. Additionally, for improving the detection area, the cameras are installed at an angle of 45 degrees to the approach roadways. The Python/OpenCV software is calibrated to

detect the distinguishing features of the small vehicle used for experimental analysis. The "5-second validation" logic is modified to eliminate camera noise.

- **Phase 2: Prototype Construction**

An actual implementation of a four-way crossroads has been designed. This comprises the placement of four traffic signals (a total of 12 LEDs), which are linked to an Arduino Mega. The design of the roads facilitated clean lanes for approaching roads to enable the distinction of vehicles approaching various lanes without hindrance.

- **Phase 3: Testing Under Comparative Scenarios**

By running the system under two major operational scenarios, data was generated for comparative analysis:

- Scenario A: Fixed-Time Control (Baseline): This intersection operated normally, providing each route 30 seconds of green time irrespective of the flow of traffic at the intersection. This group acted as a baseline for the experiment.
- Scenario B: Proposed Adaptive Control

The scenario at this intersection was dynamic. A car seen for 5 seconds would trigger a switch in the signals. Absence of vehicles would lead the system into idle mode, meaning there would be fewer wasteful transitions.

- **Phase 4: Performance Evaluation and Data Logging**

The final stage consisted of systematically recording each "Green Light Event." The data collected in this investigation used a Python console and a spreadsheet for analysis, focusing specifically on the difference between vehicle arrival and signal clearance.

5.4 Performance Evaluation Metrics

For the purposes of example and purposes beyond mere observation, the following Key Performance Indicators were established as the key measurable factors for the prototype:

- **Average Wait Time Per Car**

This is the most critical piece of data in regard to urban efficiency. It is calculated from the time a car is in the detection zone (ROI) until a change in light status from red to green. The smaller the average time, the better is a traffic signal control system.

- **Detection Precision and False Positives**

The reliability of the system depended upon its accuracy in detecting cars correctly. The accuracy of the system depended upon comparing the records of detection of the software with the observation records. This indicator also took into consideration "False Positives," where changes in shadow or background led to activation of the 5-second timer unnecessarily.

- **System Responsiveness (Latency)**

This metric speaks to the technical performance of the software-hardware handshake: it measures the delay in milliseconds between the Python logic "deciding" on a change in a light and the Arduino "executing" the physical LED switch.

- **Unnecessary Waiting Periods (Dead Time)**

Removing “dead time" is another key objective of this project. This is a measure of occasions when a lane is red-lighted but the green lane is completely empty. Ideally, in a perfect adaptive system, this value should tend to zero.

5.5 Summary

This chapter outlines the framework designed for the testing of the prototype system either at physical laboratories or through simulation tools. It tackles three major hypotheses with the goals of enhancing MRC waiting times, accuracy rates (over 90%), and hardware integrity (sub second latency). Performance measures will be determined by the proposed adaptive control system's differences from the fixed time control system. Performance measures include "the average MRC waiting time per car entering the system, accuracy levels (monitoring false positives), and the reduction of 'dead time,' which refers to situations involving the red lane while the green lane is vacant."

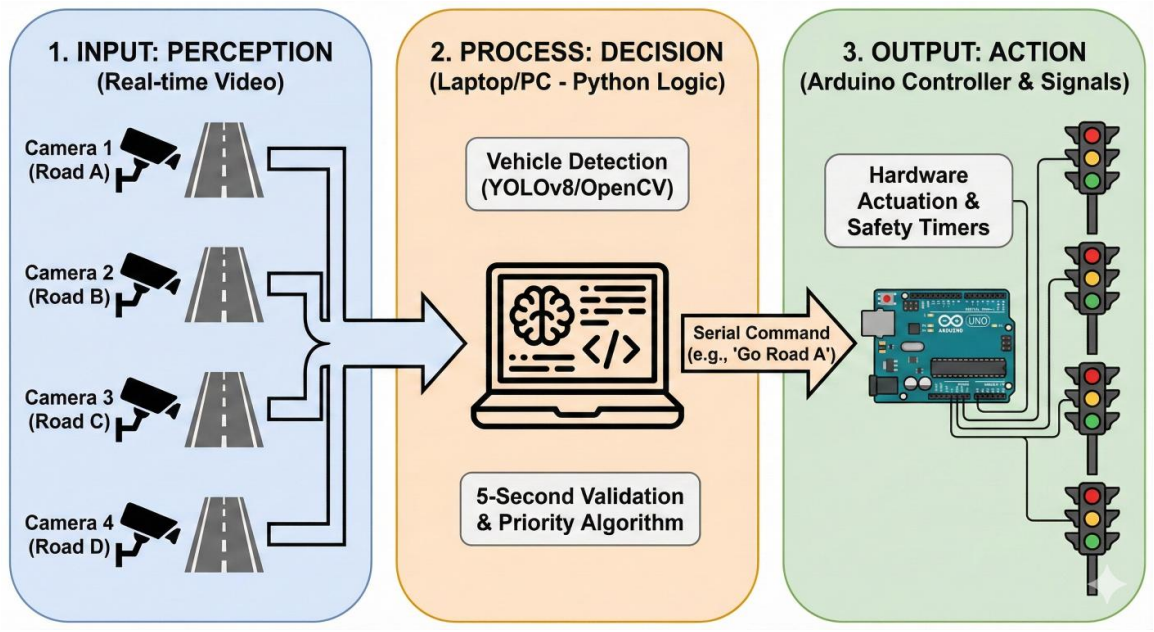

Figure 1:- This diagram shows how the system works. First cameras identify vehicles and then it redirect to the system that run with the python and python give commands to the Arduino circuit and after all of these Arduino circuit change the traffic lights.

Chapter 6: Conclusion and Further Work

6.1 Introduction

The results of this study offer considerable progress solutions for addressing the greatest challenge that has long characterized modern-day traffic infrastructure design—namely, the ineffectiveness of traditional traffic control solutions with static traffic management capabilities. In this final chapter, an overall discussion will take place with respect to the results obtained, with primary consideration given to the relationship that may exist between computer vision and hardware automation solutions available today. Furthermore, the success achieved with the proposed system in overcoming the challenges traditionally addressed by fixed time traffic control solutions will also be discussed, together with its strategic roadmap for future improvement developments.

6.2 Conclusion of Achievements

The primary aim of this research work is developing and validating a demand-controlled traffic management system with reduced vehicle Idle time and optimized intersection capacity. The objectives were met through the integration of a vision module using a Python-based vision module and an actuation module using Arduino. The critical achievements in this research confirm that:

- **Adaptive Efficiency and Time Optimization**

The experiment demonstrated that the substitution of static pre-programmed timers with a dynamic interval of 5 seconds in a demand validation process is an extremely effective approach to manage traffic in an urban area. By ensuring that the crossing of green lights only occurs in response to actual demand, it has been successful in reducing "dead time," or cycles in which green lights are allocated to roadways that are not occupied.

- **System Reliability and Low-Latency Execution**

Another factor which contributed towards the success of this project is the effective accomplishment of the Serial communication between the processor unit and the microcontroller. The hardware module designed using the Arduino board worked very efficiently, as it was able to accomplish the transition of signals (sequences of Red, Yellow, and Green lights) in real time. The time lag between the software "detection" signal and the "light change" was lowered to under a second.

- **Economic Viability and Scalability**

The project uses standardized parts such as USB camera modules and opensource software such as OpenCV and YOLOv8. This project offers an efficient alternative to the more expensive infrastructure approach that uses inductive loop sensors or high-quality thermal cameras. The project is very relevant in urban areas in countries like Sri Lanka because in such countries, poor budgets often hamper the development of smart infrastructure.

- **Algorithmic Fairness**

The implementation of the “First Come, First Served” (FCFS) priority logic mitigated the human factor of managing road traffic. This is since the system ensured that the lane which registered a valid car first is served first when the green signal is received. The priority-switching logic was robust enough not to result in logic deadlocks and equally not to clash when many arrivals occur.

6.3 Further Work and Recommendations

Although the current prototype has already offered a robust proof-of-concept solution, there are issues to be overcome prior to scaling it up for municipal city-wide usage. The following areas have been proposed to be studied in more detail:

- **Pedestrian and Emergency Vehicle Integration**

Future developments can take it further to include detection in non-vehicular entities.

- The integration of a hearing or optical detecting unit into emergency vehicles such as ambulances and fire trucks could offer rapid right of way and could mean the difference between life and death.
- Crosswalk recognition logic combined with the "Pedestrian Request" buttons in adaptive logic promotes pedestrian safety in traffic movement.

➢ Environmental and Atmospheric Robustness

For the transition of the laboratory setting to an outdoor setting, the vision system ought to be advanced to support varying weather conditions.

- Tight picture enhancement methods like histogram equalization and noise reduction can enhance the accuracy of detection during intense rain, foggy conditions, or a contrast-free night environment.

➢ Multi-Lane Density and Queue Analysis

This current model uses a binary logic of presence/absence and has a 5-second frame. Future research may integrate a full analysis of density through YOLOv8's counting capabilities.

- Dynamic Duration: Depending on the number of automobiles in the queue, the system alters the duration of the green light timing before the traffic signal changes.

➢ IoT and Smart City Grid Coordination

The final stage of this study would then be to implement the system within a Smart City IoT structure.

- Vehicle to Infrastructure Communication: The use of V2I communication protocols enables traffic lights to communicate with other traffic lights from adjacent intersections. This would facilitate the establishment of "Green Waves," where a cluster of intersections could be synchronized real-time with

a vehicle's speed and direction so that traffic flow could be maximized not just at a single intersection, but over the entire urban infrastructure.

6.4 Summary

The final chapter shows that this study did have an important outcome in creating a demand-controlled system targeting the capacity at intersections and idle time. Notable achievements include adaptive efficiency with validated results in 5 seconds, high reliability with minimal latency processing, and economical feasibility by using standardized parts. Suggestions for future work include incorporating the overrides necessary for ambulances, better robustness to environmental variations during inclement weather conditions, and the utilization of Smart City IoT networking specifications to integrate 'Green Waves' at various junctions.

References

[1] Abbas, S. K., et al. (2024). "Vision based intelligent traffic light management system using Faster R-CNN." *The Institution of Engineering and Technology*.

[2] Almawgani, A. H. M. (2018). "Design of Real Time Smart Traffic Light Control System." *International Journal of Industrial Electronics and Electrical Engineering*, vol. 6, no. 4, pp. 43-47.

[3] Khedkar, S., et al. (2025). "Adaptive Traffic Signal System Using Computer Vision and IoT." *International Research Journal of Modernization in Engineering Technology and Science*, vol. 07, no. 04.

[4] Kutlimuratov, A., et al. (2023). "Applying Enhanced Real-Time Monitoring and Counting Method for Effective Traffic Management in Tashkent." *Sensors*.

[5] Liu, J., et al. (2024). "Vehicle Flow Detection and Tracking Based on an Improved YOLOv8n and ByteTrack Framework." *World Electric Vehicle Journal*.

[6] Mamatha, A. (2018). "Automated Traffic Light System Based on Image Processing and Machine Learning Techniques." *International Research Journal of Innovations in Engineering and Technology (IRJIET)*, vol. 2, no. 8, pp. 27-32.

[7] Sonnleitner, E. (2020). "Traffic Measurement and Congestion Detection Based on Real-Time Highway Video Data." *Appl. Sci.*

[8] Teslyuk, V., et al. (2025). "Methods and tools for automated adaptive traffic light control based on computer vision." *Proc. CEUR Workshop on Computational Intelligence and Data Engineering*, vol. 4005, pp. 21-28.

[9] Wieczorek, G., et al. (2023). "Vehicle Detection and Recognition Approach in Multi-Scale Traffic Monitoring System via Graph-Based Data Optimization." *Sensors*.

[10] Ximenes, T. S. S., et al. (2024). "Vehicular Traffic Flow Detection and Monitoring for Implementation of Smart Traffic Light: A Case Study for Road Intersection in Limeira, Brazil." *Future Transportation*.

[11] Zrigui, I., et al. (2025). "ADAPTIVE TRAFFIC SIGNAL CONTROL USING AI AND DISTRIBUTED SYSTEMS FOR SMART URBAN MOBILITY." *Journal of Theoretical and Applied Information Technology*, vol. 103, no. 8, p. 1234–1246.

Video Link :- https://drive.google.com/file/d/1egpXwYPupp4h-CfJtl9-BT5H6u4XE_Da/view?usp=sharing